MESHSETU

SOS Return Channel

Acknowledgement & Distress-Relief Messaging

Complete Technical Development Bible - Flutter/Dart Implementation

Flutter/Dart mobile implementation, protocol, routing, Drift persistence, universal_ble transport, gateway/admin APIs, cURL cookbook, security, testing, observability and build plan

Version	1.1 - Flutter Edition
Prepared	25 August 2026
Current implementation target	Flutter/Dart mobile app, Android-first deployment + existing admin/server and admin/client
Feature decision	Hybrid return channel: short-lived reverse-route cache + destination-addressed controlled fallback
Status	MVP implementation specification with production-hardening path

Source precedence for this edition: the return-path design note is the feature authority; the supplied Flutter/Dart technical bible is the mobile implementation authority; context.md remains authoritative for product/protocol invariants and the live admin/server + admin/client location. Native Kotlin/Room examples in context.md are translated into Dart/Drift/universal_ble rather than copied as the implementation target.

Safety invariant: A transport ACK and an operator/application acknowledgement are different. The sender must never be told that the control room received an SOS, help was dispatched, or guidance is official unless the corresponding authority response exists and its signature verifies.

Document map

The document is intentionally implementation-oriented: field names, states, route keys, ordering rules, persistence behavior and failure semantics are specified so a developer can build the feature without having to reinterpret the product idea.

1. Scope, source authority and engineering invariants

1.1 What this document adds

This specification turns the return-path idea into a complete feature contract for the Flutter/Dart MeshSetu mobile architecture. An SOS may travel from sender A through relay phones B and C to gateway G. Each receiving relay creates a short-lived local “footprint” recording the previous peer for that SOS/origin. When an authorized admin creates an acknowledgement or guidance message, the gateway injects a signed RESPONDER_UPDATE into the BLE overlay. The update walks those reverse-route entries when available and uses bounded destination-addressed fallback when the old path has broken. [FEATURE §§4-8]

- The sender does not need Internet access. The gateway is the online/local-LAN-to-BLE bridge. **[FEATURE §1]**
- The feature reuses the existing origin_ephemeral_id, event_id, trace_id, TTL/hop limit, priorities, durable outbox, dedupe and RESPONDER_UPDATE payload type. The mobile implementation is Flutter/Dart, with Protocol Buffers generated for Dart, Drift/SQLite persistence, and universal_ble behind a narrow transport adapter. [FLUTTER BIBLE §§4-8; FEATURE §2]
- The feature does not create a permanent global path database or a long-lived user trail. “Footprints” are event-scoped, local and expiring. [FEATURE §§4,9]
- The exact original chain is a fast path, not a hard source route. A broken relay must not make the response permanently undeliverable if another path exists. **[FEATURE §6]**

1.2 Source authority and compatibility rule

Source	Role in this specification	Precedence
CURRENT - context.md	Product/protocol architecture, safety invariants, BLE/store-and-forward semantics, security intent and live admin path note.	Highest for product/protocol/admin location; platform-specific Kotlin snippets are reference only
FEATURE - return-path design DOCX	Feature decision, reverse-route semantics, fallback ladder, security/UI/test requirements.	Feature authority where not conflicting with product invariants
FLUTTER BIBLE - previous technical bible PDF	Flutter/Dart mobile implementation baseline: universal_ble, Drift, Riverpod,	Mobile implementation authority for this edition

Source	Role in this specification	Precedence
	Dart protobuf, package:http, Flutter foreground-task/platform bridge.	
Official docs / RFCs	Platform behavior and networking precedent; used to validate feasibility and implementation constraints.	External technical reference

Implementation override: This edition intentionally targets Flutter/Dart for the mobile app. Android remains the first physical-device deployment platform because the BLE peripheral/central and foreground-execution behavior must be proven there. Do not interpret older Kotlin examples as a requirement to rewrite the Flutter app natively.

1.3 Non-negotiable engineering invariants

Invariant	Implementation consequence
No false authority claim	UI copy is derived from a verified response type. SOS_RECEIVED means control-room receipt only. HELP_DISPATCHED is shown only when the authority explicitly created that type.
No false delivery claim	Admin UI enters SENDER_DELIVERED only after a sender-side delivery receipt returns.
SOS still wins	Existing scheduler order remains: CONTROL_ACK, SOS_STRUCTURED, AUTHORITY_CONTROL, VOICE_EVIDENCE, ROOM_MESSAGE, TELEMETRY. A return message must not starve a new SOS.
Route state is local and short-lived	A reverse-route entry lives only on the relay that learned it and expires no later than the related SOS/event lifetime.
Destination remains explicit	Every response carries the destination ephemeral ID even when a reverse route exists.
Dedupe and expiry are mandatory	Every response has a unique response ID/object ID, hop limit and expiry. Relays drop duplicates.
Connections are temporary	No implementation assumes the original GATT session remains open until an operator replies.
Signature before official display	Sender verifies authority signature, site, event linkage, destination and expiry before showing the message as official.
No TTS added	The authority response is displayed as verified text. This feature does not introduce Text-to-Speech; that would contradict the current product contract.
Debug path is not production identity trail	Full hop traces, if enabled for demo diagnostics, use debug-only hashed identifiers and are excluded from normal production payloads.

2. Decision summary and architecture

2.1 Recommended feature name

Use **SOS Return Channel** as the engineering feature name and **Acknowledgement & Distress-Relief Messaging** as the product-facing description. The feature includes operator acknowledgement, dispatch status, short safety guidance, incident closure status and a sender delivery receipt.

2.2 End-to-end topology

```
FORWARD SOS

Sender A  --SOS-->  Relay B  --SOS-->  Relay C  --SOS-->  Gateway G  --LAN/HTTP-->  Admin
   |                   |                 |                   |
 event/origin ID        + footprint       + footprint         + footprint
                        "came from A"     "came from B"       "came from C"

RETURN

Admin --create/sign response--> server --downlink--> Gateway G --BLE--> Relay C --BLE--> Relay B --BLE-->
Sender A
                                                                \
                                                                 +--> controlled fallback via other relays if C/B
path is gone

DELIVERY RECEIPT

Sender A --RESPONSE_DELIVERED--> normal store-and-forward / cached route --> Gateway --> server --> Admin UI
```

The “footprint” is not the entire path copied into the response. It is a local reverse-route entry on each relay. This reduces packet size, avoids exposing a complete movement trail, and makes the route repairable. **[FEATURE §4.1]**

2.3 Two acknowledgement layers

Layer	Meaning	Who consumes it	Payload/traffic class
Transport ACK	The adjacent receiver accepted a complete mesh object. It proves link/object delivery only.	BLE relay reliability machinery	ACK; CONTROL_ACK
Operator/application ACK	The control room received the SOS and generated a specific response.	Original sender and admin workflow	RESPONDER_UPDATE; AUTHORITY_CONTROL
Sender delivery receipt	The intended sender verified and displayed/accepted the authority response.	Admin/server status workflow	ACK with application receipt semantics or a dedicated receipt message; CONTROL_ACK

2.4 Why reverse cache + controlled fallback

A hard-coded source route such as G→C→B→A fails if C moves away after the SOS. The return message therefore remains destination-addressed and treats cached previous-hop state as the fastest available route. This mirrors established networking ideas: AODV creates temporary reverse next-hop state for replies; Bluetooth Mesh 1.1 directed forwarding builds path/discovery state; delay-tolerant networking uses persistent store-and-forward when a continuous path may not exist. MeshSetu does not implement those protocols verbatim; it borrows the resilience principles. [FEATURE §6, R1-R5]

3. The footprint model: short-lived reverse-route state

3.1 Footprint definition

When a relay successfully authenticates a forward SOS object, it records the peer from which that SOS arrived. The route key is **event + origin**, not a phone MAC address. The peer identity is the event/session-scoped app-level ephemeral ID learned from HELLO; a BLE address/session object may be retained only as a short-lived connection hint.

```
ReverseRouteKey = (siteId, eventId, originEphemeralId)

ReverseRouteCandidate {
    previousPeerEphemeralId   // routing identity from MeshSetu HELLO
    previousPeerHint          // short-lived BLE session/address hint; nullable
    learnedAtMs               // wall clock for expiry/persistence
    learnedAtElapsedMs        // local monotonic clock for freshness ranking, if available
    expiresAtMs               // never later than SOS expiry
    observedForwardHopCount   // lower is generally better
    lastReachableAtMs         // optional
    consecutiveFailures       // local only
    qualityScore              // optional; do not overfit for MVP
}
```

3.2 Candidate count

Keep at most **two** candidates per route key in the MVP. The preferred candidate is usually the freshest reachable previous peer with the smallest observed forward hop count. A second candidate is valuable when the same SOS arrives through another path. Additional copies remain deduped at the application layer and do not grow a topology database.

3.3 Critical receive-order change

Important integration detail: The current mesh receive sketch performs object-level dedupe early. To learn an alternate reverse candidate from a duplicate SOS that arrived through a different peer, route observation must occur after authentication/site/expiry checks but before the duplicate object is discarded for UI/re-forwarding. Candidate insertion remains capped and rate-limited. Without this ordering change, the “second route candidate” requirement can never work reliably.

```
receiveCompleteObject(ciphertext, transportMeta, fromPeer):
    envelope = decryptAndVerify(ciphertext) ?: dropInvalid()
    if expired(envelope) or wrongSite(envelope): return

    if envelope.payload_type == STRUCTURED_SOS:
        reverseRouteRepo.observeValidSos(
            siteId = envelope.site_id,
            eventId = envelope.event_id,
            originId = envelope.origin_ephemeral_id,
            fromPeer = fromPeer,
            observedForwardHopCount = envelope.hop_count,
            maxExpiry = envelope.expires_at_ms
        )

    if !recentObjectCache.markIfNew(transportMeta.objectId, envelope.expires_at_ms):
        return  // duplicate still helped route learning; do not redisplay/re-forward normally

    persistAndProcess(envelope)
```

3.4 Retention and privacy

- Expire route entries at `min(original_sos.expires_at_ms, learned_at + route_cache_ttl)`; never let a duplicate refresh the route beyond original SOS expiry.
- Do not sync relay route tables to the admin server as a normal feature.
- Do not log raw Bluetooth addresses in production telemetry. Use a short hash or event-scoped peer ID in debug logs.
- Delete expired route rows periodically and during event leave/reset.

3.5 Suggested tunables for the first implementation

These are **starting values to benchmark**, not measured product claims. Keep them in a build/config object rather than scattered constants.

Setting	Suggested start	Reason
Route cache TTL	10 min, capped by SOS expiry	Long enough for an operator to respond during a demo while still being short-lived.
Response TTL	5 min	Guidance should not circulate indefinitely; stale guidance can be unsafe.
Reverse candidates per route	2	Redundancy without topology growth.

Setting	Suggested start	Reason
Fallback fan-out	1 first, then at most 2	Limits collision/traffic amplification.
Return hop limit	6	Enough for the expected phone demo topology; keep configurable.
Max authority text	256 UTF-8 bytes	Keeps response small and fast over BLE.
Recent response cache	Expiry-bounded, target 4096 IDs	Matches current bounded-dedupe philosophy.
Gateway long poll	10-15 s request, immediate reissue	Simple dependency-light downlink without inbound phone listener.
Retry backoff	0.5s, 1s, 2s, 4s + jitter; then scan/retry cycles	Avoid synchronized storms and battery churn.

4. Repository integration and module ownership

4.1 Current target layout

```
meshsetu/
  mobile/
    lib/
      app/                  # Flutter app shell, routing, providers, active-event lifecycle
      core/model/           # Dart domain models; response/route/result types
      core/protocol/        # protobuf mapping, signatures, frame codec, fragmentation
      core/data/            # Drift DB: reverse routes + response states + durable outbox
      core/ble/             # universal_ble adapter, peer sessions, route learning, return router
      core/security/        # cryptography + secure key wrapper
      feature/sos/          # verified authority response UI + delivery receipt
      feature/gateway/      # incident uplink + authority downlink + mesh injection
    android/                # Flutter runner + narrow native bridge only when required
    pubspec.yaml
  protocol/
    meshsetu.proto          # platform-neutral source of truth; generate Dart classes
  admin/
    server/                 # live admin backend location
    client/                 # live admin frontend location
  tools/
    return_path_simulator.py
    log_parser.py
  docs/
    ADR-003-sos-return-channel.md
    RETURN_CHANNEL_RUNBOOK.md
    THREAT_MODEL.md
```

The live admin implementation remains admin/server/ + admin/client/. The mobile implementation in this edition is Flutter/Dart. Older native-Kotlin snippets from the context document are architectural references only; package/platform details must be expressed through Dart interfaces and adapters. Older dashboard/main.py FastAPI code remains an executable reference adapter unless the live repository actually uses FastAPI.

4.2 File-level responsibilities

Area	New/changed responsibility	Must not do
core/protocol	Responder update/receipt schemas; signing wrapper; payload mapping; protocol feature flag.	Do not access Bluetooth or UI.
core/data (Drift)	Reverse route table, response object state, route cleanup and retry persistence.	Do not choose BLE peers directly.
core/ble (universal_ble adapter)	Capture previous peer, reverse-route	Do not parse operator semantics beyond

Area	New/changed responsibility	Must not do
	lookup, candidate selection, controlled fallback, dedupe and retry.	routing fields.
feature/gateway (Flutter)	Receive signed commands from admin server, persist, enqueue as authority control, upload delivery receipts/status.	Do not hold the authority private signing key.
feature/sos (Flutter)	Verify destination/signature/event linkage and show official response; generate delivery receipt.	Do not show unverified text as official.
admin/server	Authorize operator, create response, sign payload, queue to ingress gateway, track states and audit.	Do not expose signing key to browser client.
admin/client	ACK/guidance UI, type-safe templates, status timeline.	Do not self-sign or claim delivered before receipt.

4.3 Proposed branch/commit boundaries

- `feat/return-protocol`: protobuf + generated code + test vectors.
- feat/reverse-route-store: Drift tables/DAO/schema migration + cleanup tests.
- `feat/reverse-route-router`: route learning + reverse selection + unit tests.
- `feat/admin-response-api`: admin/server response creation, signing, idempotency and status.
- `feat/gateway-downlink`: command poll/WebSocket adapter + mesh injection.
- `feat/sender-authority-ui`: verification, UI and delivery receipt.
- `feat/return-fallback`: alternate candidate + bounded fallback fan-out.
- `test/return-channel-e2e`: 1/2/3-hop, churn, replay, restart and priority tests.
- `docs/return-channel`: ADR, runbook, source links and measured results.

5. Wire protocol and Protocol Buffers

5.1 Compatibility strategy

The MeshEnvelope already carries event_id, site/room scope, creation/expiry, hop count/limit, priority, payload type, origin ephemeral ID and trace ID; ACK and RESPONDER_UPDATE are already reserved. Keep the envelope additive and platform-neutral. Generate Dart protobuf classes with protoc_plugin, add the responder body + signed wrapper + application delivery receipt, then reuse the existing encryption-before-fragmentation flow. [FLUTTER BIBLE §6; FEATURE §8]

5.2 Responder update: sign the exact body bytes

Do not place the signature inside the same protobuf object whose serialization is being signed. Instead, serialize a body, sign those exact bytes, and transmit those bytes plus signature in a wrapper. This avoids cross-language canonicalization ambiguity.

```
// protocol/meshsetu.proto
syntax = "proto3";
package meshsetu.v1;

enum ResponseType {
  RESPONSE_UNSPECIFIED = 0;
  SOS_RECEIVED = 1;
  HELP_DISPATCHED = 2;
  SAFETY_GUIDANCE = 3;
  INCIDENT_CLOSED = 4;
}

enum SignatureAlgorithm {
  SIG_UNSPECIFIED = 0;
  ECDSA_P256_SHA256 = 1;   // reference choice if project has not already pinned one
}

message ResponderUpdateBody {
  string response_id = 1;
  string reply_to_event_id = 2;
  string destination_ephemeral_id = 3;
```

```
  ResponseType type = 4;
  string message_text = 5;
  int64 created_at_ms = 6;
  int64 expires_at_ms = 7;
  string site_id = 8;
  bytes original_trace_id = 9;
}

message SignedResponderUpdate {
  bytes body = 1;
  bytes authority_signature = 2;
  string authority_key_id = 3;
  SignatureAlgorithm algorithm = 4;
}

// Generate with the repository-pinned protoc/protoc_plugin version:
// protoc --dart_out=mobile/lib/generated -Iprotocol protocol/meshsetu.proto
```

Algorithm compatibility: If EventManifest signing already uses a different pinned authority algorithm, reuse that algorithm instead of introducing a second one. The ECDSA_P256_SHA256 value above is a reference default for a project that has not yet frozen an authority signature primitive.

5.3 Application delivery receipt

```
enum AckKind {
  ACK_KIND_UNSPECIFIED = 0;
  OBJECT_RECEIVED = 1;              // transport/object reliability
  RESPONSE_DELIVERED = 2;           // application receipt at original sender
}

message AckMessage {
  string ack_id = 1;
  AckKind kind = 2;
  string acked_object_id = 3;       // optional transport object reference
  string response_id = 4;           // required for RESPONSE_DELIVERED
  string reply_to_event_id = 5;
  string sender_ephemeral_id = 6;
  int64 created_at_ms = 7;
  int64 expires_at_ms = 8;
}
```

Keeping `RESPONSE_DELIVERED` inside the already-reserved `ACK` payload family avoids creating another top-level `PayloadType` solely for the receipt. It must still be distinguished from transport ACKs in code and dashboard semantics.

5.4 Envelope construction for the MVP

```
final body = ResponderUpdateBody(
  responseId: responseId,
  replyToEventId: eventId,
  destinationEphemeralId: destinationId,
  type: responseType,
  messageText: messageText,
  createdAtMs: Int64(nowMs),
  expiresAtMs: Int64(expiresAtMs),
  siteId: siteId,
);

final bodyBytes = Uint8List.fromList(body.writeToBuffer());
final signature = await authoritySigner.sign(bodyBytes);
final signed = SignedResponderUpdate(
  body: bodyBytes,
  authoritySignature: signature,
  authorityKeyId: authorityKeyId,
  algorithm: SignatureAlgorithm.ECDSA_P256_SHA256,
);

final envelope = MeshEnvelope(
  eventId: body.replyToEventId,
  siteId: body.siteId,
  roomId: '',
  createdAtMs: Int64(nowMs),
  expiresAtMs: body.expiresAtMs,
  hopCount: 0,
  hopLimit: config.returnHopLimit,
  priority: Priority.P1_HIGH,
  payloadType: PayloadType.RESPONDER_UPDATE,
  payload: signed.writeToBuffer(),
  originEphemeralId: gatewayEphemeralId,
  traceId: newTraceId(),
```

```
);

final ciphertext = await crypto.encryptAndAuthenticate(envelope.writeToBuffer());
final frames = fragment(ciphertext, negotiatedMtu);
```

5.5 Production relay header (hardening path, not MVP blocker)

The current MVP can decrypt/re-encrypt at trusted site relays when hop count changes. A stronger production model keeps the signed authority body immutable and moves mutable routing state outside it. This lets relays route ciphertext without having to alter authenticated application content. [CURRENT §8.6; FEATURE §8.3]

```
message RelayHeader {
  uint32 protocol_version = 1;
  uint64 object_id = 2;
  string site_id = 3;
  string destination_ephemeral_id = 4;
  uint32 hop_count = 5;
  uint32 hop_limit = 6;
  int64 expires_at_ms = 7;
  bytes trace_id = 8;
  RouteMode route_mode = 9; // REVERSE_CACHE | ALTERNATE_CACHE | FALLBACK
}

message RoutedCiphertext {
  RelayHeader relay = 1;     // mutable, integrity-protected per hop or by transport mechanism
  bytes immutable_payload = 2; // sender/authority-signed or recipient-encrypted body
}
```

5.6 Object identity, trace identity and idempotency

- `response_id`: UUID generated once by admin/server. This is the application identity of the response.
- `object_id`: transport object identity generated once at gateway injection and preserved across relay hops. Re-encryption must not create a new object identity.
- `trace_id`: correlation ID for telemetry. The return response may get a new trace ID but carries `original_trace_id` for linkage.
- Admin API accepts an `Idempotency-Key`; repeated operator/browser retries must return the same response record instead of signing multiple duplicate responses.

6. Reverse-route persistence schema and lifecycle

6.1 Drift table

```
class ReverseRoutes extends Table {
  TextColumn get siteId => text()();
  TextColumn get eventId => text()();
  TextColumn get originEphemeralId => text()();
  TextColumn get previousPeerEphemeralId => text()();
  TextColumn get previousPeerHint => text().nullable()();
  IntColumn get learnedAtMs => integer()();
  IntColumn get expiresAtMs => integer()();
  IntColumn get observedForwardHopCount => integer()();
  IntColumn get lastReachableAtMs => integer().nullable()();
  IntColumn get consecutiveFailures => integer().withDefault(const Constant(0))();
  RealColumn get qualityScore => real().nullable()();

  @override
  Set<Column<Object>> get primaryKey => {
    siteId, eventId, originEphemeralId, previousPeerEphemeralId,
  };
}

// Add DB indexes in migration/custom SQL:
// CREATE INDEX idx_reverse_route_key
//   ON reverse_routes(site_id,event_id,origin_ephemeral_id);
// CREATE INDEX idx_reverse_route_expiry ON reverse_routes(expires_at_ms);
```

6.2 Drift DAO operations

```
@DriftAccessor(tables: [ReverseRoutes])
class ReverseRouteDao extends DatabaseAccessor<MeshDatabase>
    with _$ReverseRouteDaoMixin {
  ReverseRouteDao(super.db);

  Future<void> upsert(ReverseRoutesCompanion row) =>
      into(reverseRoutes).insertOnConflictUpdate(row);
```

```
  Future<List<ReverseRoute>> candidates(
    String siteId, String eventId, String originId, int nowMs,
  ) {
    final q = select(reverseRoutes)
      ..where((t) =>
          t.siteId.equals(siteId) &
          t.eventId.equals(eventId) &
          t.originEphemeralId.equals(originId) &
          t.expiresAtMs.isBiggerThanValue(nowMs))
      ..orderBy([
        (t) => OrderingTerm.asc(t.consecutiveFailures),
        (t) => OrderingTerm.desc(t.learnedAtMs),
        (t) => OrderingTerm.asc(t.observedForwardHopCount),
      ])
      ..limit(2);
    return q.get();
  }

  Future<int> deleteExpired(int nowMs) =>
      (delete(reverseRoutes)..where((t) => t.expiresAtMs.isSmallerOrEqualValue(nowMs))).go();
}
```

6.3 Candidate-cap enforcement

Because the composite primary key permits many peers, the repository must prune after insertion. Keep the preferred two using deterministic ordering. Do not simply use “latest two” if the newest route has repeated failures; include failure count and reachability. For a 48-hour MVP, a simple transactional insert → query all → delete losers is acceptable because each route key is tiny.

6.4 Authority response persistence

```
class AuthorityResponseOutbox extends Table {
  TextColumn get responseId => text()();
  TextColumn get replyToEventId => text()();
  TextColumn get destinationEphemeralId => text()();
  BlobColumn get signedPayload => blob()();
  IntColumn get meshObjectId => integer()();
  TextColumn get state => text()(); // READY/FORWARDING/DELIVERED/RETRY/EXPIRED/FAILED
  TextColumn get routeMode => text().nullable()();
  IntColumn get attempts => integer().withDefault(const Constant(0))();
  IntColumn get createdAtMs => integer()();
  IntColumn get expiresAtMs => integer()();

  @override
  Set<Column<Object>> get primaryKey => {responseId};
}
```

6.5 Database migration

Use a real Drift schema migration rather than destructive reset if the current build already has durable outbox state. Read the actual schemaVersion from the checked-in Flutter repository and add the return-channel tables/indexes through MigrationStrategy. Never copy a made-up database version from this document.

```
@override
int get schemaVersion => N + 1; // replace N with the repository's real current version

@override
MigrationStrategy get migration => MigrationStrategy(
  onUpgrade: (m, from, to) async {
    if (from < N + 1) {
      await m.createTable(reverseRoutes);
      await m.createTable(authorityResponseOutbox);
      await customStatement(
        'CREATE INDEX IF NOT EXISTS idx_reverse_route_key '
        'ON reverse_routes(site_id,event_id,origin_ephemeral_id)',
      );
      await customStatement(
        'CREATE INDEX IF NOT EXISTS idx_reverse_route_expiry '
        'ON reverse_routes(expires_at_ms)',
      );
    }
  },
);
```

7. Forward SOS route-learning algorithm

7.1 Preconditions

A relay must not create routing footprints for unauthenticated garbage. Learn a route only after: protocol version/size checks, complete object reassembly, decryption/authentication, site match and expiry validation. The route may be learned before application-level duplicate suppression so a duplicate valid SOS can contribute an alternate candidate.

7.2 Dart reference flow

```
Future<void> onCompleteObject(
  Uint8List ciphertext,
  TransportMeta transport,
  PeerIdentity fromPeer,
) async {
  final envelope = await crypto.decryptAndVerify(ciphertext);
  if (envelope == null) { metrics.invalidObject(); return; }

  final now = clock.wallTimeMs();
  if (now > envelope.expiresAtMs.toInt()) { metrics.expired(); return; }
  if (envelope.siteId != activeManifest.siteId) return;

  if (envelope.payloadType == PayloadType.STRUCTURED_SOS) {
    await reverseRoutes.observeValidSos(
      siteId: envelope.siteId,
      eventId: envelope.eventId,
      originId: envelope.originEphemeralId,
      fromPeer: fromPeer,
      observedForwardHopCount: envelope.hopCount,
      expiresAtMs: envelope.expiresAtMs.toInt(),
    );
  }

  final firstCopy = recentObjectCache.markIfNew(
    transport.objectId, envelope.expiresAtMs.toInt(), now,
  );
  if (!firstCopy) { metrics.duplicateObject(); return; }

  await inbox.persist(envelope);
  uiEvents.add(envelope);

  if (gatewayPolicy.isGateway &&
      envelope.payloadType == PayloadType.STRUCTURED_SOS) {
    await gatewayUplink.postIncident(envelope, ingressPeer: fromPeer);
  }

  final canRelay = envelope.hopCount < envelope.hopLimit && !gatewayPolicy.sinkOnly;
  if (canRelay) await relayNormally(envelope, excludePeer: fromPeer);
}
```

7.3 Route observation rules

- Use `origin_ephemeral_id` from the authenticated SOS envelope, never `BluetoothDevice.address`, as the route destination identity.
- Use `previousPeerEphemeralId` from the peer HELLO/session identity. The BLE address is at most a `previousPeerHint` for quick reconnection.
- Clamp route expiry to the original SOS expiry. A later duplicate may refresh reachability/freshness but may not extend beyond that original expiry.
- If the same peer repeats the same SOS, update reachability/freshness without creating a new row.
- If a second valid peer brings the same SOS, retain it as the alternate candidate if it ranks within the top two.
- If malformed or wrong-site duplicate traffic arrives, it must not modify route state.

7.4 Gateway incident metadata extension

For a server-created reply to reach the correct mesh entry point, the incident record needs fields that the older one-way dashboard may not have stored. Add these to the gateway uplink contract:

Field	Why required
origin_ephemeral_id	Target of the return message.
ingress_gateway_session_id	Which gateway received the incident; default downlink target for the reply.

Field	Why required
site_id	Server validates reply scope and authority key.
event_id	Correlation to original SOS and reverse-route key.
trace_id	Debug correlation; not a routing identity.
received_at_gateway_ms	Audit and latency evidence.
forward_hop_count	Display/metrics; not used as the only return route.

8. Return forwarding algorithm and fallback ladder

8.1 Routing decision ladder

Step	Method	When used	Behavior
1	Live previous connection	Recorded peer still has a ready GATT session	Send immediately through that session.
2	Primary reverse-route candidate	Normal case after connection ended	Reconnect/discover recorded peer using session/address hint and HELLO identity.
3	Alternate cached candidate	Primary unavailable and duplicate SOS produced a second route	Try the second candidate.
4	Controlled destination fallback	Cached route broken	Forward to 1, then at most 2 good peers; low hop limit; exclude ingress and already-attempted peers.
5	Store and retry until expiry	No route at this instant	Keep durable response and retry during later scan/connect cycles; never show delivered without receipt.

8.2 Sender delivery branch

```
if (body.destinationEphemeralId == localSession.ephemeralNodeId) {
  final verified = await authorityVerifier.verify(signedUpdate, activeManifest);
  if (verified == null) {
    metrics.authoritySignatureRejected(body.responseId);
    return;
  }
  if (verified.siteId != activeManifest.siteId) return;
  if (!localSosHistory.contains(verified.replyToEventId)) return;
  if (clock.wallTimeMs() > verified.expiresAtMs.toInt()) return;

  await authorityInbox.persistVerified(verified);
  authorityMessageController.add(verified);
  await enqueueResponseDeliveredReceipt(verified);
  return;
}
```

8.3 Relay routing branch

```
Future<void> routeAuthorityResponse(
  RoutableAuthorityResponse update,
  PeerIdentity? fromPeer,
```

```
) async {
  final now = clock.wallTimeMs();
  if (now >= update.expiresAtMs) { await expire(update); return; }
  if (!recentResponseCache.markIfNew(update.responseId, update.expiresAtMs, now)) return;
  if (update.hopCount >= update.hopLimit) { await expireHopLimit(update); return; }

  final candidates = (await reverseRoutes.candidates(
    siteId: update.siteId,
    eventId: update.replyToEventId,
    originId: update.destinationEphemeralId,
    nowMs: now,
  )).where((r) => r.previousPeerEphemeralId != fromPeer?.ephemeralId).toList();

  for (var i = 0; i < candidates.length; i++) {
    final route = candidates[i];
    final session = peerDirectory.readySession(route.previousPeerEphemeralId) ??
        await peerConnector.reconnect(
          route.previousPeerEphemeralId, route.previousPeerHint,
        );
    if (session != null && await sendToPeer(update, session)) {
      await reverseRoutes.markReachable(route);
      metrics.responseForwarded(
        update.responseId, i == 0 ? 'REVERSE_CACHE' : 'ALT_CACHE',
      );
      return;
    }
    await reverseRoutes.markFailure(route);
  }

  await controlledFallbackForward(update, exclude: fromPeer);
}
```

8.4 Controlled fallback selection

Fallback is not “flood every reply.” Select a very small set of peers from the current site. A practical MVP ranking is: ready connection first; peers not equal to ingress; peers not already attempted for this response; recent successful transfer first; stronger recent link/RSSI as a tie-breaker. Send to one peer, then a second only if needed or if policy explicitly allows parallel fan-out.

```
Future<void> controlledFallbackForward(
  RoutableAuthorityResponse msg, {PeerIdentity? exclude},
) async {
  final eligible = peerDirectory.readyPeers()
      .where((p) => p.siteFingerprint == activeSiteFingerprint)
      .where((p) => p.ephemeralId != exclude?.ephemeralId)
      .where((p) => !responseAttempts.wasTried(msg.responseId, p.ephemeralId))
      .toList()
    ..sort((a, b) {
      final connected = (b.isConnected ? 1 : 0).compareTo(a.isConnected ? 1 : 0);
      if (connected != 0) return connected;
      final recent = b.lastTransferSuccessMs.compareTo(a.lastTransferSuccessMs);
      if (recent != 0) return recent;
      return (b.lastRssi ?? -999).compareTo(a.lastRssi ?? -999);
    });

  final selected = eligible.take(config.fallbackFanoutMax).toList();
  if (selected.isEmpty) {
    await responseOutbox.markRetry(msg.responseId);
    return;
  }
  for (final peer in selected) {
    responseAttempts.markTried(msg.responseId, peer.ephemeralId);
    await sendToPeer(msg.copyWith(routeMode: ReturnRouteMode.fallback), peer);
  }
}
```

8.5 Loop and amplification controls

- Every response has one stable `response_id` and one stable transport `object_id` across hops.
- Relay drops a response already present in its recent-response cache.
- Increment hop count and stop at hop limit or expiry.
- Never fallback to the peer that just sent the response unless that peer is itself the destination and the response is being locally delivered.
- Cap attempted peers per response and retain that attempt state only until response expiry.
- Rate-limit authority-control injection per operator/site to avoid a compromised admin account overwhelming the mesh.
- Fallback fan-out is not increased without measured collision/delivery evidence.

9. BLE/GATT transport behavior

9.1 Reuse the existing GATT service

No new BLE service is required for the return channel. The Flutter implementation keeps one MeshSetu GATT service and wraps universal_ble behind core/ble. Authority response objects use the same object encryption, fragmentation, reassembly and ACK mechanisms as other application objects. The rest of the app never depends directly on plugin-specific BLE types. [FLUTTER BIBLE §§2.3,4.2,7]

```
const meshServiceUuid = 'PROJECT-SERVICE-UUID';
const meshRxUuid      = 'PROJECT-RX-UUID';   // peer writes frames
const meshTxUuid      = 'PROJECT-TX-UUID';   // notifications/control
const meshCtrlUuid    = 'PROJECT-CTRL-UUID'; // HELLO/ACK/inventory

abstract interface class MeshBleAdapter {
  Stream<PeerAdvertisement> get discoveries;
  Future<void> startAdvertising(Uint8List siteFingerprint);
  Future<void> startScan();
  Future<BlePeerSession> connect(String deviceId);
  Future<int?> requestMtu(String deviceId, int requested);
  Future<void> writeFrame(BlePeerSession peer, Uint8List frame);
}

// UniversalBleMeshAdapter is the only layer that imports universal_ble.
```

9.2 MTU and frame sizing

The frame header remains 16 bytes. The Flutter BLE adapter requests/observes the effective ATT MTU through universal_ble where supported and passes the resulting value to the fragmenter. Never hard-code 517 bytes or any plugin default. Android remains the first deployment target, so underlying Android GATT behavior still applies even though product code is Dart. [FLUTTER BIBLE §2.3; Android BluetoothGatt docs]

```
ATT application value ~= negotiatedMtu - 3
MeshSetu fragment payload = max(0, negotiatedMtu - 3 - FRAME_HEADER_BYTES)
FRAME_HEADER_BYTES = 16

Example only:
  MTU 185 -> ~166 response payload bytes per MeshSetu frame
  MTU 247 -> ~228 response payload bytes per MeshSetu frame
  MTU 23  -> only ~4 bytes after this header; technically possible but inefficient
```

9.3 HELLO capability

Add a capability bit for the return channel if mixed-version nodes are possible. Example: `CAP_RETURN_CHANNEL = 1 << n`. A peer advertising/HELLOing protocol support does not reveal personal identity; the node ID remains event/session ephemeral. For the hackathon, update all demo phones together and fail visibly if an incompatible protocol version is encountered.

9.4 Connection lifecycle

- Treat BLE/GATT sessions as temporary. Cache MeshSetu peer identity separately from universal_ble connection/device state.
- If the previous plugin device/session hint still resolves, reconnect quickly. If it does not, re-scan with universal_ble and use the MeshSetu HELLO ephemeral ID to recover the route identity.
- Do not keep a radio link open for minutes solely because an SOS was forwarded through that peer.
- Persist response object before transmission. Connection loss returns the object to retry state.
- Run scanning, reconnect, downlink polling and retry under the Flutter active-event lifecycle. Use flutter_foreground_task or a narrow Android platform-service bridge only when required for background BLE continuity on the tested devices; durable Drift state remains mandatory. [FLUTTER BIBLE §§1.2,17]

9.5 Advertising privacy note

The current advertising sketch exposes only a service UUID and compact site fingerprint; the ephemeral node ID is learned in HELLO after connection. That is compatible with this design. The short-lived BLE address/session hint may fail if the OS rotates addresses; when that happens, the router falls through to discovery/fallback. Do not expand advertising with stable device identifiers just to make reverse routing easier.

10. Gateway downlink and admin/server integration

10.1 Gateway is bidirectional

The gateway already uplinks verified incidents to the admin side. This feature adds the downlink half: server -> Flutter gateway client -> durable Drift outbox -> BLE mesh. The gateway does not author authority messages and must not possess the server authority private signing key. It validates basic scope/signature, persists the command, then injects it as AUTHORITY_CONTROL. [FEATURE §10.3]

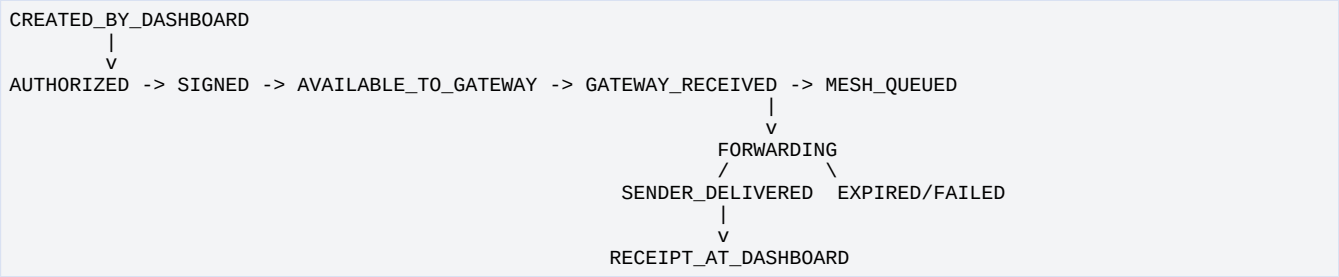
10.2 Why the gateway should initiate the admin connection

For an Android phone running the Flutter gateway on a hotspot/LAN, an inbound HTTP listener is unnecessary complexity. A dependency-light MVP is for Dart code to use package:http for a short/long-poll GET to admin/server while active event mode is running. A WebSocket client is also valid if the live admin/server already exposes one and the Flutter app already has a tested client. The feature contract is transport-neutral.

10.3 Required admin/server records

Record	Minimum fields
Incident	event_id, site_id, origin_ephemeral_id, ingress_gateway_session_id, trace_id, received_at, forward hop count, current status
Authority response	response_id, event_id, destination_ephemeral_id, response_type, message_text or protected content reference, signed_payload, key_id, created/expires, operator_id, state, idempotency_key
Gateway command	response_id, gateway_session_id, queue cursor/sequence, available_at, received_at, state
Delivery receipt	receipt_id, response_id, event_id, sender_ephemeral_id, gateway_received_at, receipt blob/hash, verification state

10.4 Response state machine



`SENDER_DELIVERED` is not set by “gateway sent it.” It is set only when the sender’s application receipt returns. `RECEIPT_AT_DASHBOARD` can be used as the terminal audit state once the server ingests the receipt.

10.5 Gateway failure behavior

- If admin/server is unreachable, the gateway continues normal BLE mesh operation and retains its local state. New admin replies cannot enter the mesh until the downlink returns.
- If gateway receives a signed command but BLE is temporarily unavailable, it persists the response and shows `MESH_QUEUED/RETRY`, not failure.
- If the original ingress gateway is unavailable, MVP server keeps the response pending. Production may fan the signed command to another authorized gateway on the same site, with dedupe by response ID.

11. REST API contract and cURL cookbook

11.1 API design principles

- Browser/client requests an action; **server** performs authorization and signing.
- `response_id` is server-generated; operator retries use `Idempotency-Key`.
- Gateway downlink returns already-signed payload bytes. Gateway never signs as authority.
- All APIs return machine-readable states. The UI does not infer delivery from HTTP 200 alone.
- Hackathon may reuse the existing `X-MeshSetu-Demo-Key`; production must use real operator/gateway authentication and audited authorization.

11.2 Create an authority response

```
POST /api/events/{event_id}/responses
Idempotency-Key: <uuid>
Content-Type: application/json
Authorization: <existing admin auth>      # production
X-MeshSetu-Demo-Key: <demo key>           # hackathon only, if current server uses it

{
  "type": "SOS_RECEIVED",
  "message_text": "Your SOS has reached the control room.",
  "expires_in_seconds": 300
}

201 Created
{
  "response_id": "6bb5...",
  "reply_to_event_id": "c22a...",
  "destination_ephemeral_id": "evt-ephemeral-...",
  "state": "SIGNED",
  "ingress_gateway_session_id": "gw-session-...",
  "created_at_ms": 1787600000000,
  "expires_at_ms": 1787600300000
}
```

11.3 Gateway command fetch (recommended MVP long poll)

```
GET /api/gateways/{gateway_session_id}/commands?cursor=<cursor>&wait_ms=15000&limit=20

200 OK
{
  "cursor": "00000129",
  "commands": [
    {
      "response_id": "6bb5...",
      "site_id": "event-2026",
      "payload_type": "RESPONDER_UPDATE",
      "signed_payload_b64": "...",
      "expires_at_ms": 1787600300000
    }
  ]
}
```

11.4 Gateway queue receipt

```
POST /api/gateways/{gateway_session_id}/commands/{response_id}/received
Content-Type: application/json

{
  "mesh_object_id": "81726354011223344",
  "received_at_ms": 1787600001450
}

200 OK
{"state":"GATEWAY_RECEIVED"}
```

11.5 Upload sender delivery receipt

```
POST /api/responses/{response_id}/receipts
Content-Type: application/json

{
  "gateway_session_id": "gw-session-...",
  "receipt_payload_b64": "...",
  "gateway_received_at_ms": 1787600019120
}
```

```
200 OK
{"state":"RECEIPT_AT_DASHBOARD"}
```

11.6 Query response status

```
GET /api/responses/{response_id}

200 OK
{
  "response_id": "6bb5...",
  "state": "SENDER_DELIVERED",
  "route_mode": "REVERSE_CACHE",
  "return_hops": 3,
  "retry_count": 0,
  "created_at_ms": 1787600000000,
  "gateway_injected_at_ms": 1787600001450,
  "sender_delivery_receipt_at_ms": 1787600019120
}
```

11.7 cURL cookbook

```
# Variables
export BASE="http://<ADMIN_SERVER_LAN_IP>:8080"
export EVENT_ID="<event-id>"
export GW="<gateway-session-id>"
export KEY="<hackathon-demo-key>"

# 1) Create control-room receipt acknowledgement
curl -i -X POST "$BASE/api/events/$EVENT_ID/responses" \
  -H "Content-Type: application/json" \
  -H "X-MeshSetu-Demo-Key: $KEY" \
  -H "Idempotency-Key: $(python -c 'import uuid; print(uuid.uuid4())')" \
  -d '{"type":"SOS_RECEIVED","message_text":"Your SOS has reached the control room.","expires_in_seconds":300}'

# 2) Ask for pending gateway commands (short form; live server may long-poll)
curl -s "$BASE/api/gateways/$GW/commands?wait_ms=15000&limit=20" \
  -H "X-MeshSetu-Demo-Key: $KEY"

# 3) Mark one signed command received by gateway
curl -i -X POST "$BASE/api/gateways/$GW/commands/<response-id>/received" \
  -H "Content-Type: application/json" \
  -H "X-MeshSetu-Demo-Key: $KEY" \
  -d '{"mesh_object_id":"81726354011223344","received_at_ms":1787600001450}'

# 4) Query state
curl -s "$BASE/api/responses/<response-id>" \
  -H "X-MeshSetu-Demo-Key: $KEY"
```

API path adaptation: The exact endpoint prefixes must be mapped onto the current admin/server routing conventions. The contract and state semantics are the important part. Do not replace working live server architecture with FastAPI solely because an older technical bible used FastAPI examples.

12. Flutter/Dart reference implementation

12.1 Domain types

```
enum ReturnRouteMode { liveSession, reverseCache, altCache, fallback, retry }

final class RoutableAuthorityResponse {
  const RoutableAuthorityResponse({
    required this.responseId,
    required this.replyToEventId,
    required this.destinationEphemeralId,
    required this.siteId,
    required this.createdAtMs,
    required this.expiresAtMs,
    required this.hopCount,
    required this.hopLimit,
    required this.signedPayload,
    required this.meshObjectId,
  });
  final String responseId;
  final String replyToEventId;
  final String destinationEphemeralId;
  final String siteId;
  final int createdAtMs;
  final int expiresAtMs;
  final int hopCount;
```

```
  final int hopLimit;
  final Uint8List signedPayload;
  final int meshObjectId;
}

final class ForwardResult {
  const ForwardResult({required this.accepted, this.routeMode, this.peerEphemeralId, this.detail});
  final bool accepted;
  final ReturnRouteMode? routeMode;
  final String? peerEphemeralId;
  final String? detail;
}
```

12.2 ReverseRouteRepository

```
final class ReverseRouteRepository {
  ReverseRouteRepository(this.dao, this.clock, this.config);
  final ReverseRouteDao dao;
  final Clock clock;
  final ReturnChannelConfig config;

  Future<void> observeValidSos({
    required String siteId,
    required String eventId,
    required String originId,
    required PeerIdentity fromPeer,
    required int observedForwardHopCount,
    required int expiresAtMs,
  }) async {
    final now = clock.wallTimeMs();
    final routeExpiry = min(expiresAtMs, now + config.routeCacheTtlMs);
    if (routeExpiry <= now) return;

    await dao.upsert(ReverseRoutesCompanion.insert(
      siteId: siteId,
      eventId: eventId,
      originEphemeralId: originId,
      previousPeerEphemeralId: fromPeer.ephemeralId,
      previousPeerHint: Value(fromPeer.connectionHint),
      learnedAtMs: now,
      expiresAtMs: routeExpiry,
      observedForwardHopCount: observedForwardHopCount,
      lastReachableAtMs: Value(now),
    ));
    await pruneToBestTwo(siteId, eventId, originId, now);
  }
}
```

12.3 Sender signature verification

The EventManifest already carries an authority public key in the current architecture. Verification must use that pinned event/site key, not a key supplied inside the response.

```
final class AuthorityResponseVerifier {
  AuthorityResponseVerifier(this.authorityKeys, this.clock);
  final AuthorityKeyStore authorityKeys;
  final Clock clock;

  Future<ResponderUpdateBody?> verifyAndParse(
    SignedResponderUpdate signed, EventManifest manifest,
  ) async {
    final publicKey = await authorityKeys.publicKey(
      manifest.authorityPublicKeyB64, signed.authorityKeyId,
    );
    if (publicKey == null) return null;

    final ok = await verifyAuthoritySignature(
      publicKey: publicKey,
      algorithm: signed.algorithm,
      body: Uint8List.fromList(signed.body),
      signature: Uint8List.fromList(signed.authoritySignature),
    );
    if (!ok) return null;

    final body = ResponderUpdateBody.fromBuffer(signed.body);
    if (body.siteId != manifest.siteId) return null;
    if (clock.wallTimeMs() > body.expiresAtMs.toInt()) return null;
    if (utf8.encode(body.messageText).length > 256) return null;
    return body;
  }
}
```

12.4 Delivery receipt generation

```
Future<void> enqueueDeliveryReceipt(ResponderUpdateBody body) async {
  final now = clock.wallTimeMs();
  final receipt = AckMessage(
    ackId: const Uuid().v4(),
    kind: AckKind.RESPONSE_DELIVERED,
    responseId: body.responseId,
    replyToEventId: body.replyToEventId,
    senderEphemeralId: localSession.ephemeralNodeId,
    createdAtMs: Int64(now),
    expiresAtMs: Int64(min(body.expiresAtMs.toInt(), now + config.receiptTtlMs)),
  );
  await outbox.enqueue(
    payloadType: PayloadType.ACK,
    trafficClass: TrafficClass.controlAck,
    eventId: body.replyToEventId,
    payload: Uint8List.fromList(receipt.writeToBuffer()),
  );
}
```

12.5 Gateway downlink poller

```
final class AuthorityDownlinkPoller {
  AuthorityDownlinkPoller(this.server, this.responseStore, this.meshOutbox);
  final AdminServerClient server;
  final AuthorityResponseStore responseStore;
  final MeshOutbox meshOutbox;

  Future<void> run(String gatewaySessionId, CancellationToken cancel) async {
    String? cursor;
    while (!cancel.isCancelled) {
      try {
        final batch = await server.pollCommands(
          gatewaySessionId, cursor, waitMs: 15000,
        );
        cursor = batch.cursor;
        for (final cmd in batch.commands) {
          if (cmd.isExpired(clock.wallTimeMs())) continue;
          if (!await responseStore.insertIfNew(cmd.responseId, cmd.signedPayload)) continue;
          final objectId = await meshOutbox.enqueueAuthorityControl(cmd);
          await server.markGatewayReceived(
            gatewaySessionId, cmd.responseId, objectId,
          );
        }
      } catch (_) {
        await Future<void>.delayed(backoff.nextWithJitter());
      }
    }
  }
}
```

12.6 Foreground service integration

Start the downlink poller and BLE coordinator from the Flutter active-event controller/provider. If the process must continue while the UI is backgrounded on Android, run only the required coordination through flutter_foreground_task or a narrow platform-channel/Pigeon bridge. Stop it when event mode ends, keys are revoked, or the manifest expires.

12.7 Dart interfaces and Riverpod boundary

```
abstract interface class ReturnRouter {
  Future<void> learnFromForwardSos(MeshEnvelope envelope, PeerIdentity fromPeer);
  Future<ForwardResult> route(
    RoutableAuthorityResponse update, {PeerIdentity? fromPeer}
  );
}

abstract interface class AdminServerClient {
  Future<CommandBatch> pollCommands(
    String gatewaySessionId, String? cursor, {required int waitMs}
  );
  Future<void> markGatewayReceived(
    String gatewaySessionId, String responseId, int objectId
  );
  Future<void> uploadDeliveryReceipt(String responseId, Uint8List receipt);
}

// Riverpod wires concrete Drift/BLE/HTTP implementations at the app boundary.
// core/ble is the only mobile module that imports universal_ble.
```

Flutter screens and feature controllers depend on repositories/interfaces, not universal_ble, Drift internals, or admin framework details. The BLE layer does not know how the admin browser is built; the admin server does not know GATT internals. feature/gateway remains the boundary.

13. Admin/server reference implementation

13.1 Framework-neutral behavior

1. Authenticate and authorize the operator.
2. Load incident by `event_id`; require `origin_ephemeral_id`, site and ingress gateway.
3. Validate response type and message length/semantics.
4. Enforce idempotency.
5. Create `ResponderUpdateBody`, serialize exact bytes, sign on server, persist response.
6. Queue signed bytes for the incident ingress gateway session.
7. Publish state update to admin client (WebSocket/SSE/poll, according to live stack).
8. Accept gateway receipt and sender delivery receipt; transition state monotonically.

13.2 Reference FastAPI adapter (illustrative only)

```
# Reference adapter only. Port the contract into the live admin/server framework.
from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel, Field
import time, uuid

app = FastAPI()

class CreateResponse(BaseModel):
    type: str
    message_text: str = Field(min_length=1, max_length=256)
    expires_in_seconds: int = Field(default=300, ge=30, le=900)

@app.post("/api/events/{event_id}/responses", status_code=201)
def create_response(event_id: str, req: CreateResponse, idempotency_key: str = Header(alias="Idempotency-Key")):
    existing = repo.find_by_idempotency(idempotency_key)
    if existing:
        return existing.public_view()

    incident = repo.get_incident(event_id)
    if not incident:
        raise HTTPException(404, "event not found")

    authorize_response_type(req.type, incident)
    now = int(time.time() * 1000)
    response_id = str(uuid.uuid4())
    body_bytes = build_responder_body(
        response_id=response_id,
        event_id=event_id,
        destination_ephemeral_id=incident.origin_ephemeral_id,
        site_id=incident.site_id,
        response_type=req.type,
        message_text=req.message_text,
        created_at_ms=now,
        expires_at_ms=now + req.expires_in_seconds * 1000,
    )
    signed = authority_signer.sign_wrapper(body_bytes, incident.site_id)
    record = repo.create_response_and_gateway_command(
        response_id=response_id,
        idempotency_key=idempotency_key,
        incident=incident,
        signed_payload=signed,
    )
    publish_state(record)
    return record.public_view()
```

13.3 Signing-key rule

Private key placement: The authority private key belongs in server-side secret storage or an external signing service. It must never be bundled in admin/client, exposed in browser JavaScript, copied into Flutter assets/Dart source, or printed in logs.

13.4 Type-specific authorization

Response type	Server-side rule	Safe default copy
SOS_RECEIVED	Incident exists and was ingested by	Your SOS has reached the control room.

Response type	Server-side rule	Safe default copy
	control room. May be operator-triggered or policy-triggered after ingestion.	
HELP_DISPATCHED	Require explicit dispatch state/authorized operator action.	Help has been dispatched.
SAFETY_GUIDANCE	Require authorized operator; enforce short bounded text and audit.	Context-specific; no generic promise.
INCIDENT_CLOSED	Require incident closure permission/state.	This incident has been closed by the control room.

14. Admin/client behavior and operator safeguards

14.1 Incident card additions

- `Acknowledge SOS` button that creates `SOS_RECEIVED`.
- Response-type selector with role-gated options.
- Safe templates plus a short free-text field for `SAFETY_GUIDANCE`.
- Status timeline: `SIGNED → GATEWAY_RECEIVED → MESH_QUEUED/FORWARDING → SENDER_DELIVERED → RECEIPT_AT_DASHBOARD`, with failure/expiry states.
- Display `event_id` and `response_id` in an expandable technical panel for audit/debug.
- Display route mode (`reverse`, `alternate`, `fallback`) only as technical telemetry; do not expose relay identities to ordinary operators.
- Display “Pending mesh delivery” rather than “Delivered” until sender receipt returns.

14.2 Sender UI additions

Condition	Sender UI
Verified SOS_RECEIVED	Official control-room message: “Your SOS has reached the control room.”
Verified HELP_DISPATCHED	Official message: “Help has been dispatched.”
Verified SAFETY_GUIDANCE	Show exact bounded authority guidance, source label and timestamp.
Expired response	Do not show as current guidance; log/retain only according to policy.
Bad signature/wrong site/wrong destination	Discard as official content; debug metric only.
Duplicate response	Show once; receipt may be retransmitted idempotently if needed.

14.3 Accessibility and notification behavior

Use a distinct “Control room” label, response type and timestamp. A Flutter local-notification integration may alert the user, but this feature must not introduce Text-to-Speech. Notification text must be the same verified content stored in the app, with sensitive lock-screen display following platform privacy settings.

15. Cryptographic security and privacy

15.1 Threats specific to the return channel

Threat	MVP mitigation	Production hardening
Fake authority guidance	Authority signature; sender verifies against EventManifest public key.	Hardware/HSM-backed authority key, key rotation and audited trust chain.
Replay of an old valid response	Response ID cache + expiry + event/site/destination checks.	Persistent replay windows/counters where appropriate.
Response misdelivery	Explicit destination ephemeral ID + event linkage.	Sender-only encryption with ephemeral reply key.
Route poisoning	Learn only from authenticated valid SOS; cap two candidates; expiry; failure scoring.	Origin signatures, per-device credentials, route-error/local-repair logic.
Relay eavesdropping	Existing site transport encryption.	Encrypt response inner body to sender-only ephemeral key.
Mesh amplification	Low hop limit, bounded fan-out, dedupe, response size/rate limits.	Adaptive quotas/abuse detection.
Breadcrumb privacy leak	Local short-lived route entries; no permanent server path trail.	Strict retention/audit, encrypted local DB if threat model requires.
Compromised browser	Browser cannot sign; server authorizes/signs.	Strong operator auth, CSRF protection, device posture, audit and revocation.

15.2 Signature coverage

The signature covers the exact `ResponderUpdateBody` bytes, including response ID, event ID, destination, response type, message text, site and expiry. **Do not sign mutable routing fields** such as hop count or route mode. Because the signed body bytes are carried verbatim, every language verifies the exact byte sequence that the server signed.

15.3 Reference ECDSA verification

```
Future<bool> verifyEcdsaP256({
  required EcdsaPublicKey publicKey,
  required Uint8List body,
  required Uint8List signatureBytes,
}) async {
  // Implement through the project-pinned cryptography provider/package.
  // Keep this adapter covered by server<->Dart golden test vectors.
  return authorityCrypto.verify(
    publicKey: publicKey,
    message: body,
    signature: signatureBytes,
    algorithm: AuthorityAlgorithm.ecdsaP256Sha256,
  );
}
```

Use the project's already-pinned authority algorithm if different. Avoid inventing home-grown signatures or using a shared symmetric site key as proof of "official control-room" authorship when the architecture already provisions an authority public key.

15.4 Sender-only confidentiality (production)

The MVP site transport key may allow trusted site relays to decrypt response content. For private guidance, the sender can include an ephemeral reply public key in the SOS. The authority encrypts the inner responder body to that key and then signs the resulting wrapper; relays route destination-addressed ciphertext without reading the guidance. Pin the Dart cryptography implementation and underlying Android support with cross-language test vectors before enabling this production path.

15.5 Key lifecycle

- EventManifest contains authority public key/key ID and is verified before event activation.
- Server maps `site_id` to the correct signing key; response creation fails closed if no valid key is available.
- Flutter app accepts only authority keys trusted by the active EventManifest.
- Store locally cached site/room secrets through flutter_secure_storage (Android-backed Keystore where supported) or a reviewed platform secure-storage adapter; never hard-code long-lived secrets in Dart source or assets. [FLUTTER BIBLE §16]
- On event expiry/leave, clear short-lived route state and event secrets according to existing key-retention policy.

16. Persistence, background execution and lifecycle

16.1 Durable-first rule

Every admin response is persisted in Drift before BLE transmit. Every reverse-route observation is persisted before it is relied upon after a process restart. This follows the Flutter architecture where Drift/SQLite is the durable outbox/inbox, not merely UI storage. [FLUTTER BIBLE §§2.5,17]

16.2 Response outbox states

State	Meaning	Transition trigger
READY	Signed response exists locally and is eligible for forwarding.	Gateway command persisted.
FORWARDING	At least one peer send attempt is active/just occurred.	Router chooses a peer.
RETRY	No current route or transient send failure.	No eligible peer / disconnect / NACK.
DELIVERED	Sender-side receipt observed locally at gateway or uploaded to server.	Receipt object returns.
EXPIRED	Wall-clock expiry passed or hop limit exhausted with no valid route.	Cleanup/router check.
FAILED	Permanent validation/configuration failure.	Invalid command/signature/site or policy error.

16.3 Process restart behavior

9. Flutter active-event mode restarts/re-enters according to the existing app lifecycle and foreground-task policy.
10. Load non-expired route entries and response outbox rows.
11. Clear stale universal_ble connection/device hints; never assume plugin connection objects survive process death.
12. Resume scan/connect cycles.
13. Retry READY/RETRY responses in scheduler order.
14. Delete expired route and response rows and emit expiry metrics.

16.4 Foreground-service constraints

Android still imposes the underlying BLE background constraints even when the app is Flutter. Keep the Flutter active-event experience user-visible and use flutter_foreground_task or a narrow native connected-device service bridge where required. The return channel must depend on persisted Drift objects plus discovery/reconnect/retry, not on a permanent BLE connection. [FLUTTER BIBLE §17; Android BLE background docs]

17. Observability, trace IDs and measurement correctness

17.1 Required metric events

Metric kind	When emitted	Core fields
reverse_route_learned	Valid SOS teaches a previous peer.	event_id, hashed_peer, hop_count, expiry
reverse_route_duplicate_candidate	Duplicate valid SOS adds/refreshes alternate peer.	event_id, candidate_count
response_created	Admin/server persisted signed response.	response_id, event_id, type
gateway_command_received	Gateway persisted server command.	response_id, gateway_session
response_forwarded	Relay sent response to a peer.	response_id, route_mode, hashed_peer, hop
reverse_route_miss	No usable cached candidate.	response_id, event_id
fallback_forwarded	Controlled fallback used.	response_id, fanout, hop
response_duplicate_drop	Duplicate response suppressed.	response_id
authority_signature_rejected	Sender/relay rejected bad signature.	response_id/key_id if parseable
response_delivered	Sender verified response and queued receipt.	response_id, event_id
delivery_receipt_gateway	Gateway received sender receipt.	response_id, return hops
response_expired	Response aged out.	response_id, last_route_mode, attempts

17.2 Privacy-safe logging

Do not put raw guidance, full transcripts, site keys, platform BLE device identifiers or personal notes in logs. Use response/event IDs and short hashed peer identifiers. In Flutter, write structured debug metrics to app-private storage rather than relying on console logs for the final demo evidence. [FLUTTER BIBLE §17]

17.3 Clock correctness

Measurement nuance: Wall-clock timestamps across multiple phones can be skewed. In Dart use DateTime.now().millisecondsSinceEpoch for expiry/human timestamps and Stopwatch for local monotonic durations. The most trustworthy return-channel latency metric is gateway-injection-to-delivery-receipt round trip measured on the same gateway process/monotonic clock. One-way gateway-to-sender latency requires synchronized clocks and must be labeled approximate unless synchronization is demonstrated.

17.4 Demo metrics

- SOS origin-to-gateway latency (with clock-method caveat).
- Forward hop count.
- Dashboard response creation time.
- Gateway injection time.
- Gateway-to-receipt round-trip latency.
- Return hop count.
- Route mode: reverse cache / alternate / fallback.
- Retry count and duplicate drops.

- Final state: delivered / expired / pending.
- Priority-preemption evidence while voice/room traffic is active.

18. Failure modes and recovery matrix

Failure	System behavior	User/admin truth
Original relay moved away	Try second cached candidate, then bounded fallback, then persist/retry until expiry.	“Reply is being forwarded” / “Pending mesh delivery.”
No route now	Persist response; scan/retry later.	Never claim delivered.
Sender process unavailable	Relays keep response within TTL and retry when topology changes.	Admin remains pending.
Duplicate response copies	Dedupe by stable response/object ID.	Sender shows one official message.
Bad authority signature	Drop and log metric.	Nothing official is shown.
Replay after expiry	Reject by expiry/recent-response cache.	Nothing current is shown.
Gateway loses server LAN	Mesh continues; downlink pauses.	Admin shows gateway/downlink offline.
Gateway gets command but BLE off	Persist command; retry after Bluetooth restored.	Pending mesh delivery.
App restart on relay	Reload route/outbox; connection hints revalidated; retry.	No false delivered.
Low MTU	More fragments; response remains feasible; prioritize over voice.	No user-visible protocol detail unless debug.
Voice transfer active	Authority response is higher priority than voice and should preempt remaining chunks after current safe write boundary.	Authority update arrives without waiting for full voice transfer.
Admin duplicate click/retry	Idempotency key returns same response record.	One logical response timeline.

19. Test plan, failure injection and acceptance gates

19.1 Unit tests

- Route insertion uses `(site,event,origin)` and previous peer ephemeral ID, not BLE address as identity.
- Duplicate valid SOS through second peer yields at most two candidates.
- Duplicate same-peer SOS refreshes candidate without adding rows.
- Expired/wrong-site/bad-auth SOS never modifies reverse route.
- Candidate ordering prefers reachable/nonfailed/fresh route deterministically.
- Response destination branch verifies signature before UI.
- Wrong destination is never displayed.
- Replay/duplicate response is shown once.
- Hop limit and expiry stop forwarding.
- Ingress peer exclusion prevents immediate bounce loops.
- Fallback fan-out never exceeds configured maximum.
- Priority queue places new structured SOS ahead of authority control and authority control ahead of voice/room.
- Delivery receipt is `CONTROL_ACK` class and links response/event IDs.
- Admin response type guard prevents `HELP_DISPATCHED` without dispatch state.
- Idempotency key returns same response ID on retry.

19.2 Drift/integration tests

- Drift migration creates reverse-route indexes/tables and retains existing durable outbox data.
- Kill relay process after learning route; restart within TTL; route row is available and stale connection object is not assumed.
- Expiry cleanup removes route and response rows.
- Flutter active-event coordinator/foreground task resumes without duplicate command injection.

19.3 Physical BLE system tests

Test	Setup	Pass condition
1-hop ACK	Sender A → gateway G → admin; reply back.	A verifies signed response and receipt reaches server.
2-hop reverse	A → B → G; Internet off on A/B.	G → B → A using cached reverse route.
3-hop reverse	A → B → C → G.	Reply follows cached reverse path; duplicates suppressed.
Broken original relay	After SOS, move C out of range; add D with alternate reachability.	Fallback delivers via another path or remains pending/expiry without false delivered.
Duplicate forward path	SOS reaches gateway through two relay paths.	One admin incident; route caches hold bounded alternate candidates.
Priority under load	Transfer voice + 20 room messages, then create authority response/new SOS.	New SOS first; authority response ahead of remaining voice/room.
Process restart	Kill/restart B after route learned.	Persisted route/object resumes or falls back; no crash/false state.
Bluetooth toggle	Turn BLE off/on on one relay.	Retry/recovery is logged and response remains bounded by expiry.
Low MTU	Force/test minimum realistic negotiated MTU on a device pair.	Response fragments/reassembles or cleanly remains pending; no overflow.
Gateway LAN outage	Disconnect gateway from admin LAN after SOS.	Mesh remains; response queue resumes when LAN returns.

19.4 Security tests

- Change one byte in signed body -> signature verification fails.
- Change signature bytes -> fails.
- Reuse valid signed response for different `site_id`/event/destination -> fails validation.
- Replay after `expires_at_ms` -> rejected.
- Inject unsigned fake guidance -> never appears as official.
- Attempt oversized message -> server rejects before signing; Flutter app also enforces a defensive UTF-8 byte cap.
- Operator without dispatch permission attempts `HELP_DISPATCHED` -> server rejects.
- Reuse Idempotency-Key with different payload -> server must reject conflict or return original; must not silently create a second response.

19.5 Suggested hackathon acceptance gates

These are engineering gates, not public reliability claims. Replace them with measured targets after trials.

- 10 consecutive static 2-hop return-channel runs succeed with no duplicate sender display.
- A 3-hop run succeeds and route mode/hops are visible in telemetry.
- Broken-path test either reaches sender via controlled fallback or remains pending/expired without a false delivered state.
- Bad-signature and replay tests produce zero official sender messages.
- Authority response preempts active voice/room transfer according to scheduler order.

- Kill/restart one relay and demonstrate durable state behavior within route TTL.
- Final report records actual median/P95/attempt metrics where sample count is sufficient; no “instant” or “99.9%” claim without data.

20. Build sequence, commit boundaries and three-person execution plan

20.1 Vertical-slice order

Phase	Deliverable
Phase 0 - Freeze contracts	Freeze proto fields, response semantics, route key, config defaults, admin API shape and test vectors before parallel coding.
Phase 1 - Direct reverse path	ResponderUpdate + signing wrapper; reverse-route Drift table; route learned on SOS; admin ACK API; gateway command injection; sender verify/display.
Phase 2 - Delivery receipt + observability	Application receipt, state timeline, metrics, cURL/runbook, response status API.
Phase 3 - Alternate candidate + fallback	Learn duplicate-path candidate before dedupe return; second candidate; bounded fallback fan-out; retry persistence.
Phase 4 - Failure hardening	Process restart, BLE toggle, wrong signature, replay, idempotency, low MTU, gateway LAN outage.
Phase 5 - Demo instrumentation	Judge-facing timeline, route mode, hops, latency, retries, final state and safe templates.

20.2 Three-person split

Person	Primary work	Integration contract
Developer A - Mesh/Protocol	Proto additions, Drift route schema with Dev B review, route learning, reverse router, fallback, dedupe/hop/TTL tests, universal_ble physical tests.	Exposes Dart ReturnRouter and metrics; accepts signed payload bytes + route-key fields.
Developer B - App/Gateway/Admin integration	Flutter app/gateway integration: Riverpod wiring, package:http downlink, response outbox, sender UI/receipt, admin/server API + admin/client controls.	Implements Dart AdminServerClient; never bypasses core/ble adapter or changes plugin internals from feature code.
Developer C - Security/Test/Integration or ML dev borrowing integration time	Authority signature test vectors, threat tests, failure injector, E2E harness, log parser, process-restart tests, demo measurements.	Publishes deterministic fixtures and expected outcomes; no hidden protocol forks.

20.3 Integration checkpoints

- Checkpoint A: server signs one response; Dart verifier passes a shared golden test vector.
- Checkpoint B: one phone learns a reverse route row from a synthetic valid SOS.
- Checkpoint C: gateway injects signed response into local outbox and transmits one hop.
- Checkpoint D: two-hop physical reverse path reaches sender and returns receipt.
- Checkpoint E: broken path invokes alternate/fallback without duplicate UI.
- Checkpoint F: demo dashboard state matches actual sender receipt and measured logs.

21. Migration/lineage from the previous technical bible

21.1 What remains architecturally consistent

The supplied Flutter/Dart technical bible is the mobile implementation baseline for this edition: Flutter UI and orchestration, Drift persistence, universal_ble for BLE central/peripheral behavior, Dart protobuf generation, Riverpod boundaries, package:http for the gateway bridge, and narrow Android platform bridges only where plugin/background behavior requires them. The return channel extends that architecture rather than replacing it.

21.2 Native-Android material that is architectural reference only

Legacy item	Current rule
Native Kotlin mobile modules / Room / direct BluetoothGatt code	Translate to Flutter/Dart. Use Drift for persistence and universal_ble behind core/ble; keep native Android code only as a narrow platform bridge when proven necessary.
dashboard/ FastAPI sample as if it were live code	Live code remains admin/server/ + admin/client/. Port API semantics into the existing server framework.
Kotlin/Java protobuf generation commands	Use Dart protobuf generation with protoc_plugin and keep meshsetu.proto platform-neutral.
Android foreground service called directly from feature code	Use Flutter active-event coordination; flutter_foreground_task or a narrow Pigeon/platform-channel bridge owns Android service details.
Direct Android Keystore calls from Dart feature code	Use flutter_secure_storage or a reviewed secure-storage adapter; keep platform key-store details behind core/security.

Native Kotlin/Room/BluetoothGatt snippets in the context document explain protocol intent and Android platform constraints, but they are not the implementation target for this edition. Translate them into Dart contracts, Drift persistence and the universal_ble adapter; only use Kotlin through a narrow platform channel/Pigeon bridge when a tested Flutter package cannot expose the required Android behavior.

21.3 Reusable ideas from the previous build

- Protocol remains bearer-independent: application envelope is not a full Bluetooth SIG Mesh claim.
- Durable local database is the queue.
- Gateway communicates with admin side through a narrow contract.
- Measure physical-device latency/reliability instead of relying on emulator assumptions.
- Use official documentation as source of truth and pin tested versions in the repository.
- Keep network/model/UI modules isolated so parallel work does not collapse into one integration bottleneck.

22. Production hardening roadmap

Area	MVP	Production direction
Routing identity	Event/session ephemeral ID + short BLE hint.	Cryptographically bound per-session identity and stronger route provenance.
Route repair	2 cached candidates + bounded fallback.	On-demand route discovery/local repair and route error feedback.
Payload confidentiality	Site transport encryption.	Sender-only response encryption; relays route ciphertext.
Origin/authority authenticity	Authority signature on response; shared site trust may exist for relays.	Separate origin signatures from mutable relay metadata; device credentials.
Gateway redundancy	Ingress gateway first.	Multiple gateways with signed command dedupe and site-aware selection.

Area	MVP	Production direction
Admin auth	Existing hackathon/demo auth where applicable.	Strong operator auth/RBAC, audit, revocation, CSRF/session hardening.
Route storage	Drift/SQLite under app sandbox + short TTL.	Encrypted at rest if threat model requires; stricter deletion/audit.
Transport	Application-layer BLE overlay.	Fixed relays may adopt standards-compliant Bluetooth Mesh directed forwarding where suitable.
Disruption tolerance	Persist/retry to response expiry.	More explicit DTN status/report semantics and custody-like policy only if requirements justify complexity.
Formal protocol	Feature bit + protobuf.	Versioned compatibility matrix, golden test vectors, fuzzing and protocol conformance suite.

23. Definition of done

- [ ] Protocol compiles and all new protobuf fields are documented with backward-compatibility rules.
- [ ] Reverse route table persists `(site,event,origin) -> up to 2 previous peers` and expires correctly.
- [ ] Valid duplicate SOS can teach an alternate route before object-level dedupe returns.
- [ ] Admin/server can create a response idempotently, authorize response type and sign it without exposing private key to browser/client.
- [ ] Gateway receives signed command, persists it, injects it as `AUTHORITY_CONTROL`, and reports queue receipt.
- [ ] 1-hop, 2-hop and 3-hop reverse-path tests pass on physical Android phones running the Flutter app.
- [ ] Broken-path test exercises alternate or controlled fallback; no uncontrolled flooding occurs.
- [ ] Sender verifies signature, site, event, destination and expiry before official display.
- [ ] Sender creates a `RESPONSE_DELIVERED` application receipt; admin does not show delivered before that receipt.
- [ ] Process restart and Bluetooth toggle do not corrupt state or create duplicate sender messages.
- [ ] Replay and forged response tests fail safely.
- [ ] New SOS retains higher priority than authority response; authority response retains higher priority than voice/room traffic.
- [ ] Logs capture response ID, route mode, hops, retries and final state without storing route breadcrumbs or sensitive content unnecessarily.
- [ ] cURL runbook can create response, inspect gateway queue and inspect final state on the live admin/server implementation.
- [ ] Official documentation links and source traceability are checked into docs/; final demo report uses measured values only.

Appendix A. Reference snippets

A.1 ReturnChannelConfig

```
final class ReturnChannelConfig {
  const ReturnChannelConfig({
    this.routeCacheTtlMs = 10 * 60 * 1000,
    this.responseTtlMs = 5 * 60 * 1000,
    this.receiptTtlMs = 5 * 60 * 1000,
    this.reverseCandidateLimit = 2,
    this.fallbackFanoutMax = 2,
    this.returnHopLimit = 6,
    this.authorityTextMaxBytes = 256,
  });
  final int routeCacheTtlMs;
  final int responseTtlMs;
  final int receiptTtlMs;
  final int reverseCandidateLimit;
  final int fallbackFanoutMax;
  final int returnHopLimit;
  final int authorityTextMaxBytes;
}
```

A.2 Response type safety mapping

```
bool allowedText(ResponseType type, String text, IncidentRecord incident, Operator operator) {
  final bytes = utf8.encode(text);
  if (bytes.isEmpty || bytes.length > 256) return false;
  return switch (type) {
    ResponseType.SOS_RECEIVED => incident.receivedAtControlRoomMs != null,
    ResponseType.HELP_DISPATCHED => incident.dispatchState == DispatchState.dispatched,
    ResponseType.SAFETY_GUIDANCE => operator.canSendGuidance,
    ResponseType.INCIDENT_CLOSED => operator.canClose && incident.isClosed,
    _ => false,
  };
}
```

A.3 Route-cache cleanup worker

```
Future<void> cleanupReturnChannel([int? nowMs]) async {
  final now = nowMs ?? clock.wallTimeMs();
  await reverseRouteDao.deleteExpired(now);
  await authorityResponseDao.expireBefore(now);
  recentResponseCache.prune(now);
  responseAttempts.prune(now);
}
```

A.4 Unit test: duplicate learns alternate route

```
test('duplicate SOS can learn second reverse candidate without duplicate UI', () async {
  final sos = validSos(eventId: 'E1', originId: 'A', objectId: 42);
  await receiver.onCompleteObject(sos.ciphertext, transport(objectId: 42), peer('B'));
  await receiver.onCompleteObject(sos.ciphertext, transport(objectId: 42), peer('D'));

  final routes = await dao.candidates(SITE, 'E1', 'A', now());
  expect(routes.map((r) => r.previousPeerEphemeralId).toSet(), {'B', 'D'});
  expect(ui.displayedSosCount('E1'), 1);
});
```

A.5 Unit test: response destination and signature

```
test('forged authority response is never displayed', () async {
  final signed = validSignedResponse(destination: localEphemeralId).tamperBodyByte();
  final parsed = await verifier.verifyAndParse(signed, manifest);
  expect(parsed, isNull);
  expect(ui.officialMessages, isEmpty);
});
```

A.6 Unit test: fallback bounded

```
test('fallback never exceeds configured fanout', () async {
  peerDirectory.setReadyPeers([for (var i = 1; i <= 10; i++) peer('P$i')]);
  await router.route(responseWithoutCachedRoute(), fromPeer: peer('P1'));
  expect(transport.sentDistinctPeers.length <= config.fallbackFanoutMax, isTrue);
  expect(transport.sentDistinctPeers.contains('P1'), isFalse);
});
```

A.7 Example protocol telemetry JSONL

```
{"time_ms":1787600001000,"event_id":"E1","kind":"reverse_route_learned","peer":"p#7ab3","value":1}
{"time_ms":1787600001500,"event_id":"E1","kind":"response_forwarded","detail":"REVERSE_CACHE","value":1}
{"time_ms":1787600019100,"event_id":"E1","kind":"delivery_receipt_gateway","value":3,"detail":"hops"}
```

Appendix B. Command and debugging cheat sheet

```
# Flutter / Android build and test
flutter doctor -v
flutter devices
flutter pub get
dart run build_runner build --delete-conflicting-outputs
flutter analyze
flutter test
flutter run -d <DEVICE_ID>
flutter build apk --debug
adb logcat | grep -i meshsetu

# Dart Protocol Buffers
protoc --dart_out=mobile/lib/generated -Iprotocol protocol/meshsetu.proto
# Prefer the repository-pinned protoc/protoc_plugin versions.

# BLE debugging through the Flutter adapter + Android platform logs
adb shell dumpsys bluetooth_manager

# Admin server
# Start using the live admin/server instructions. Older FastAPI commands are reference-only.

# Return-channel log filtering (or parse the app-private JSONL metrics file)
adb logcat | grep -E "reverse_route|response_forwarded|fallback|response_delivered|authority_signature"
```

Appendix C. Source traceability matrix

Requirement/decision	Primary local source	This document
BLE app-layer store-and-forward, not certified phone Bluetooth Mesh	CURRENT §§1.2,2.2,7-8; LEGACY §§1-2	§§2,9
RESPONDER_UPDATE and ACK already reserved	CURRENT §6.1; FEATURE §2	§5
Authority response can return through gateway	CURRENT §3.4/§15.4; FEATURE §§1,5	§§2,10-14
Short-lived reverse route + fallback	FEATURE §§4-7,12,19	§§3,7-8
Do not require exact old hop chain	FEATURE §§1,6	§8
Authority signature and sender verification	FEATURE §9; CURRENT §16	§15
Route privacy / ephemeral IDs	FEATURE §§4,9; CURRENT §§8.1,16.5	§§3,15
Durable outbox and retry	CURRENT §§2.5,8.7,17; FEATURE §§5,13-15	§§6,16,18-19
Priority over voice/room	CURRENT §8.2; FEATURE §2/§15	§§1,19
Live admin paths are admin/server + admin/client	CURRENT page-1 note	§§4,10-14
Flutter/Dart is the mobile implementation target	FLUTTER BIBLE cover + §§1-7; this edition override	§§4,6-9,12,16,20-21

Appendix D. Official documentation and research references

Flutter documentation: https://docs.flutter.dev/ - framework, build, testing and platform integration.

Flutter platform channels: https://docs.flutter.dev/platform-integration/platform-channels - narrow Android escape hatch when package APIs are insufficient.

Dart native interoperability: https://dart.dev/interop/c-interop - FFI boundary if native crypto/audio/STT helpers are required elsewhere in the project.

universal_ble package: https://pub.dev/packages/universal_ble - Flutter BLE central/peripheral package used behind core/ble; pin the tested version.

Drift package: https://pub.dev/packages/drift and drift_flutter: https://pub.dev/packages/drift_flutter - durable typed SQLite persistence and migrations.

Riverpod: https://riverpod.dev/ - application boundary/dependency orchestration for Flutter features and repositories.

flutter_foreground_task: https://pub.dev/packages/flutter_foreground_task - Android-first active-event background/foreground coordination; verify on every demo device.

flutter_secure_storage: https://pub.dev/packages/flutter_secure_storage and cryptography: https://pub.dev/packages/cryptography - secure local key wrapper and reviewed Dart crypto primitives; keep authority private keys server-side.

Protocol Buffers for Dart: https://pub.dev/packages/protobuf and https://pub.dev/packages/protoc_plugin - generated Dart protocol classes and code generation.

Bluetooth Mesh Directed Forwarding: https://www.bluetooth.com/mesh-directed-forwarding/ - Path discovery, relay state and resilience precedent; not a claim that the phone overlay implements SIG Mesh.

Bluetooth Mesh Protocol specification: https://www.bluetooth.com/specifications/specs/mesh-protocol/ - Standards reference for Bluetooth Mesh.

RFC 3561 - AODV: https://www.rfc-editor.org/rfc/rfc3561.html - Architectural precedent for temporary reverse routes; RFC is Experimental.

RFC 9171 - Bundle Protocol Version 7: https://www.rfc-editor.org/rfc/rfc9171.html - Destination-addressed store-and-forward and status concepts for disruption-tolerant networks.

RFC 4838 - DTN Architecture: https://www.rfc-editor.org/info/rfc4838/ - Persistent store-and-forward architecture for disrupted networks.

FastAPI WebSockets: https://fastapi.tiangolo.com/advanced/websockets/ - reference only if the live admin/server uses or adopts FastAPI/WebSockets.

Appendix E. Assumptions and open decisions

Decision	Status	Action before merge
Authority signature algorithm	Not explicitly frozen in supplied current context.	Reuse existing EventManifest algorithm; otherwise pin ECDSA P-256/SHA-256 (or another reviewed choice) with cross-language test vectors.
Live admin/server framework and persistence store	Not supplied in attachments.	Implement the API/state contract in existing admin/server; do not replace framework unnecessarily.
Exact Drift schema version	Not supplied.	Use actual Flutter repository schemaVersion for the migration.
Exact response text byte cap/TTL/hop limit	This document provides benchmark starting values.	Freeze after physical-device trials and product safety review.
Gateway downlink transport	Recommended long-poll for dependency-light MVP; WebSocket is valid if live stack already has it.	Choose one and test under LAN reconnect/process restart.
Sender-only encryption	Production hardening, not required for first MVP.	Pin crypto suite/provider only after compatibility/security review.

Decision	Status	Action before merge
Multiple gateways	MVP uses incident ingress gateway first.	Add multi-gateway fan-in/fan-out only after dedupe and trust model are defined.

Final implementation rule: When a behavior in this document conflicts with the checked-in repository, do not silently “fix” the repo from this document. First update the ADR/spec or adjust this document so the source-of-truth contract and implementation match. The return channel is safety-adjacent; undocumented divergence is a defect.

END OF TECHNICAL BIBLE